SenYummies - Contrat Backend (API + BD)

29 octobre 2026

Contents

Chez Ketchup / SenYummies — Contrat Backend (API + Base de données) 1

1. Entités et schéma de données . . . 2
   - 1.1 Commande (entité centrale) . . . 2
   - 1.2 LigneCommande . . . 2
   - 1.3 Zone de livraison . . . 3
   - 1.4 Plat du jour & Accompagnement / Condiment (fiches indépendantes) . . . 3
   - 1.5 Client (sans compte) . . . 4
2. Endpoints REST . . . 4
   - Produits & catalogue . . . 4
   - Plat du jour . . . 4
   - Accompagnements & condiments . . . 5
   - Zones de livraison . . . 5
   - Commandes . . . 5
   - Nouveautés / promos . . . 6
   - Zones — note de compatibilité . . . 6
3. Récapitulatif des décalages encore ouverts (hors périmètre de la correction UI/UX déjà livrée) . . . 6
4. Corrections déjà livrées côté Client_RestoApp (pour mémoire, aucun impact backend) 7
5. Écarts avec le contrat `contrat-api/openapi.yaml` actuel (vérifié le 29/10/2026) 7

Chez Ketchup / SenYummies — Contrat Backend (API + Base de données)

Document de synthèse à l'intention de l'équipe Backend (`Backend_RestoApp`), produit après un audit croisé de `Client_RestoApp` et `Manager_RestoApp` (UI/UX uniquement, aucun code backend touché). Objectif : donner un schéma de données et une liste d'endpoints suffisamment précis pour démarrer l'implémentation, en résolvant explicitement les points où les deux apps front actuelles divergent.

Statut au 29/10/2026 : aucun backend réel n'existe encore. Les deux apps front tournent sur des données mock (`Manager_RestoApp` : modules JS en mémoire ; `Client_RestoApp` : `db.json` + `json-server`, et quelques données statiques en dur). Ce document décrit ce que le backend doit implémenter pour que les deux apps puissent s'y brancher sans modification de leur modèle de données actuel (côté Manager) ou avec un minimum d'adaptation documentée (côté Client).

---

1. Entités et schéma de données

1.1 Commande (entité centrale)

Les deux apps ont aujourd'hui des modèles de Commande incompatibles. Le modèle ci-dessous reprend la structure de Manager (plus mature, déjà alignée sur un vrai cycle de vie avec rôles) et y ajoute ce que Client a besoin d'exposer (prix figés, coordonnées client complètes).

```
Commande
├─ id                  string (uuid ou identifiant lisible, ex. "CMD-2026-0142")
├─ statut          enum: CREEE | PAYEE | EN_PREPARATION | PRETE | EN_LIVRAISON | LIVREE | RETIREE | ANNU
├─ canal           enum: APP | COMPTOIR      -- APP = passée via Client_RestoApp, COMPTOIR = saisie par
├─ mode            enum: LIVRAISON | RETRAIT  -- RETRAIT = retrait comptoir ; aucun mode "sur place" (di
in) n'existe
├─ moyenPaiement       enum: wave | orange-money | mixx-yas | carte | especes
│                 -- "especes" n'est pertinent qu'au comptoir (canal=COMPTOIR) : une commande APP
│                          est payée en ligne avant sa création, donc jamais en espèces.
├─ statutPaiement      enum: en_attente | paye | echoue
├─ clientPrenom         string | null
├─ clientNom            string | null
├─ clientTelephone    string | null  -- obligatoire si canal=APP (sert de clé de retrouvaille sans com
├─ adresseLivraison   string | null  -- obligatoire si mode=LIVRAISON ; voir 1.3 pour le détail zone/s
├─ note            string | null  -- instructions libres du client ("Sonner à l'interphone B12", "Sans
├─ creeLe               datetime (ISO 8601)
├─ lignes[]             LigneCommande[]
├─ sousTotal            number          -- somme des lignes, hors livraison
├─ fraisLivraison       number          -- 0 si mode=RETRAIT
├─ total                number
├─ livreur                    { nom, telephone, photo, note, vehicule } | null    --
rempli uniquement si mode=LIVRAISON et statut >= EN_LIVRAISON
```

Points tranchés par rapport à l'état actuel des deux apps :

- **Prix sur la ligne** : Manager ne stocke aujourd'hui aucun prix sur `lignes[]` (recalcul à la volée depuis le catalogue Produits), alors que Client envoie un prix figé par ligne et des totaux déjà calculés. **Décision retenue pour le backend : snapshotter le prix unitaire sur chaque ligne à la création de la commande** (voir `LigneCommande` ci-dessous). C'est indispensable pour l'intégrité d'un historique de commandes : si le prix d'un produit change après coup, une commande passée ne doit pas voir son total recalculé rétroactivement. Manager devra s'adapter pour lire `prixUnitaire` depuis la commande plutôt que de le relire dans le catalogue — actuellement non implémenté dans aucune des deux apps de façon cohérente.
- **Nom client** : Client collecte un champ `nom` unique (pas de prénom/nom séparés) ; Manager attend `clientPrenom` + `clientNom` séparément (voir ses données de démo `cuisineDemo.js`). **Le formulaire de checkout de Client_RestoApp devra être adapté** (champ prénom + nom séparés, ou une règle de split côté backend — déconseillé, fragile). Non corrigé dans cette passe (hors périmètre "UI/UX pure", nécessite une décision de l'équipe Client).
- **Adresse précise manquante côté Client** : le formulaire de checkout ne collecte qu'une zone + un quartier (prix de livraison), jamais de complément d'adresse (bâtiment, étage, repère). Manager, lui, a une démo avec un champ `note` libre qui sert à ça ("Sonner à l'interphone B12"). **Recommandation : exposer dans le formulaire de checkout un champ "note / précisions de livraison" optionnel**, qui existe déjà dans le modèle Manager mais n'est jamais rempli côté Client aujourd'hui.

1.2 LigneCommande

```
LigneCommande
├─ produitId            string
├─ nom                  string            -- snapshot du nom au moment de la commande
```

```
├─ prixUnitaire      number        -- snapshot du prix au moment de la commande (voir 1.1)
├─ quantite            number
├─ personnalisation     string | null     -- ex. "Sans oignon", "Bien cuit"
├─ accompagnementNom    string | null     -- nom de l'accompagnement choisi (voir 1.4)
```

1.3 Zone de livraison

Décision retenue (demande explicite) : le modèle de référence est celui de Manager_RestoApp, pas celui de Client. Les deux structures actuelles :

- **Manager** (`zonesLivraisonData.js`) : `Région → { nomArrondissement: prix }`, une seule région active par défaut (Dakar), activable/désactivable par région, plus des zones personnalisées ajoutées/masquées par le Gérant (`zonesPersonnalisees` / `zonesSupprimees`), référentiel officiel des arrondissements dans `senegalZones.js`.
- **Client** (`db.json`) : `Zone → { quartiers: [{nom, prix}] }`, noms et granularité différents de Manager (ex. “Plateau” = une zone avec 5 quartiers côté Client, alors que “Plateau/Gorée” est un seul arrondissement côté Manager).

Schéma backend retenu (sur le modèle Manager) :

```
Region
├─ code                string (ex. "DAKAR")
├─ nom                 string
├─ active          boolean       -- une région peut être activée/désactivée par le Gérant

Secteur              -- "arrondissement" chez Manager ; remplace "quartier" chez Client
├─ id                  string
├─ regionCode          string (FK → Region)
├─ nom                 string
├─ prixLivraison       number
├─ estOfficiel      boolean       -- false si ajouté manuellement par le Gérant (zonesPersonnalisees
├─ estMasque       boolean       -- true si un secteur officiel a été masqué (zonesSupprimees), sans
```

Conséquence pour l’équipe Client : le sélecteur zone→quartier du checkout (`Confirmation.jsx`) devra être réécrit pour consommer `Region`/`Secteur` à la place de son `db.json` actuel (noms de lieux différents, structure à un seul niveau au lieu de deux). Ce n’est pas fait dans cette passe — à prévoir dès que l’endpoint `/zones` existe réellement.

1.4 Plat du jour & Accompagnement / Condiment (fiches indépendantes)

Décision déjà actée côté Manager (24/10/2026) : ces deux familles sont des **fiches indépendantes**, pas des entrées du catalogue Produits.

```
PlatDuJour
├─ id
├─ nom
├─ prix
├─ photo               string | null     -- optionnel
├─ moment              enum: dejeuner | diner
├─ statut              enum: BROUILLON | ACTIF | ARCHIVE

AccompagnementCondiment
├─ id
├─ nom
├─ prix                number            -- 0 autorisé (ex. sauces gratuites)
├─ type                enum: ACCOMPAGNEMENT | CONDIMENT
├─ statut              enum: BROUILLON | ACTIF | ARCHIVE
```

Écart constaté côté Client (non corrigé dans cette passe, documenté ici pour le backend et l'équipe Client) : - La section "Plats du jour" de l'accueil (`SectionPlatsDuJour.jsx`) lit des données 100 % statiques (`data/produits.js`, `categorie: 'plats-du-jour'`), sans aucun appel API — complètement déconnectée de ce que le Gérant publie désormais dans Manager. - Le panier (`Panier.jsx`) propose des "condiments" en les tirant du catalogue Produits générique (`getProduits()` filtré sur `categorie === 'condiments'`), pas de la nouvelle ressource dédiée. - **Une fois `/plats-du-jour` et `/accompagnements-condiments` exposés par le backend, Client devra rebrancher ces deux écrans dessus.**

1.5 Client (sans compte)

Pas d'entité "Utilisateur" avec mot de passe — confirmé par le README de Client_RestoApp ("Jamais de compte obligatoire"). Le téléphone sert de clé de retrouvaille :

```
GET /commandes?telephone=XXXXXXXXX   -- retrouve l'historique sans authentification
```

À noter : cet endpoint existe déjà côté contrat Client (`api/commandes.js` → `getCommandesParTelephone`) mais n'est jamais appelé par l'app (MesCommandes.jsx utilise un mock local) — câblage à faire côté Client une fois le backend prêt, hors périmètre de cette passe (choix explicite : on ne branche pas l'API maintenant, cf. décision du 29/10).

---

2. Endpoints REST

Base : `/api/v1` (voir `.env.example` de Client, `VITE_API_BASE_URL`). Format d'erreur standard déjà documenté côté Client et maintenant correctement consommé par `api/client.js` : `{ code, message, field }`.

Produits & catalogue

Méthode	Route	Description
GET	`/produits`	Liste du catalogue (hors Plat du jour / Accompagnements). Filtre `?categorie=` avec les valeurs réelles : `burgers`, `accompagnements`, `condiments`, `desserts`, `boissons`.
GET	`/produits/{id}`	Détail d'un produit.
POST / PUT / DELETE	`/produits...`	CRUD Gérant (déjà modélisé côté Manager, `menuData.js`).

Plat du jour

Méthode	Route	Description
GET	`/plats-du-jour`	Liste des plats du jour `ACTIF` (filtrable par `moment=dejeuner|diner`). Client doit consommer cette route au lieu de son mock.
POST / PUT / DELETE	`/plats-du-jour...`	CRUD Gérant.

Accompagnements & condiments

Méthode	Route	Description
GET	`/accompagnements-condiments`	Liste des fiches `ACTIF` (filtrable par `type=ACCOMPAGNEMENT|CONDIMENT`).
POST / PUT / DELETE	`/accompagnements-condiments...`	CRUD Gérant.

Zones de livraison

Méthode	Route	Description
GET	`/regions`	Liste des régions, avec `active`.
GET	`/regions/{code}/secteurs`	Liste des secteurs d'une région (prix inclus).
POST/PUT/DELETE	`/regions/{code}` et `/secteurs/{id}`	Activation région, ajout/masquage de secteur (Gérant).

Commandes

Méthode	Route	Description
POST	`/commandes`	Création (canal=APP depuis Client, canal=COMPTOIR depuis Vendeur). Statut initial `PAYEE` si `canal=APP` (paiement en ligne obligatoire avant création) ou `CREEE`/`PAYEE` selon le flux comptoir.
GET	`/commandes?telephone=`	Historique client (sans compte).
GET	`/commandes?statut=&canal=`	Files de travail Cuisine / Livreur / Vendeur.
PATCH	`/commandes/{id}/statut`	Transition de statut. Le backend **doit** revalider la transition côté serveur (403 si non autorisée) — voir la table ci-dessous, déjà implémentée en lecture seule côté Manager (`transitions.js`) à des fins d'affichage uniquement.

Table des transitions autorisées (à faire respecter côté backend) :

De	Vers	Rôle(s)	Mode requis
PAYEE	EN_PREPARATION	CUISINE	—
EN_PREPARATION	PRÊTE	CUISINE	—

De	Vers	Rôle(s)	Mode requis
PRETE	EN_LIVRAISON	CUISINE ou LIVREUR	LIVRAISON
EN_LIVRAISON	LIVREE	LIVREUR	LIVRAISON
PRETE	RETIREE	VENDEUR	RETRAIT
PAYEE	ANNULEE	CUISINE	— (uniquement depuis PAYEE : une fois la préparation commencée, plus d'annulation)

Nouveautés / promos

Méthode	Route	Description
GET	`/nouveautes?active=true`	Déjà consommé correctement par Client (`NotifContext.jsx`).

Zones — note de compatibilité

Tant que `/regions + /secteurs` ne sont pas prêts, Client peut continuer sur son `db.json` local (zones→quartiers) pour le développement, mais **la structure définitive à cibler est celle de la section 1.3**, pas celle du `db.json` actuel.

---

3. Récapitulatif des décalages encore ouverts (hors périmètre de la correction UI/UX déjà livrée)

Ces points ne sont **pas** des bugs purement front — ils nécessitent soit une décision produit, soit l'existence du backend pour être résolus proprement. Listés ici pour que l'équipe Backend les ait en tête en construisant le contrat.

1. **Modèle de zones** : Client devra réécrire son sélecteur zone/quartier sur `Region`/`Secteur` (voir 1.3) — changement de structure et de noms de lieux.
2. **Prix sur les lignes de commande** : Manager devra arrêter de recalculer le prix depuis le catalogue et lire le `prixUnitaire` snapshotté sur la commande (voir 1.1/1.2).
3. **Plat du jour / Accompagnements côté Client** : actuellement 100 % statique (Plat du jour) ou piochés dans le catalogue générique (condiments) — à rebrancher sur `/plats-du-jour` et `/accompagnements-condiments` une fois disponibles.
4. **Accueil/Menu/MesCommandes de Client n'appellent pas l'API** pour les produits et l'historique de commandes (fonctions déjà prêtes côté `api/`, jamais utilisées) — câblage repoussé volontairement (décision du 29/10/2026) tant que le backend n'est pas stable.
5. **Nom client en un seul champ** côté formulaire de checkout (`nom`) vs `clientPrenom`/`clientNom` séparés côté Manager — à harmoniser quand ce formulaire sera retravaillé.
6. **Absence de champ "note/instructions de livraison"** dans le checkout Client, alors que le modèle de commande le supporte déjà (utilisé par Manager côté Cuisine/Livreur) — amélioration UX à prévoir.

---

4. Corrections déjà livrées côté Client_RestoApp (pour mémoire, aucun impact backend)

Référence : commit `fix(ui): corrections UI/UX suite a l'audit croise Client/Manager` sur la branche `feature/audit-ui-ux-client`.

- Couleur de marque Tailwind corrigée (`#A6192E` au lieu de `#E63946`).
- Reconnaissance complète des statuts de commande dans l'écran de suivi (`PRETE`, `RETIREE`, `ANNULEE`, `PAYEE`/`CREEE` en plus des 4 déjà gérés).
- Suppression de 10 fichiers orphelins jamais importés (composants/hooks d'un ancien écran de suivi, remplacés depuis par `MesCommandes.jsx`), plus un `db.json` dupliqué et obsolète.
- Normalisation du mode de commande sur `livraison`/`retrait` (aligné sur `LIVRAISON`/`RETRAIT` de Manager).
- Clés des moyens de paiement alignées sur Manager (`orange-money`, `mixx-yas`).
- `api/client.js` : variable d'environnement corrigée (`VITE_API_BASE_URL`) et conservation de `code`/`field` sur les erreurs API.
- Catégories du catalogue mock alignées sur celles réellement utilisées par l'API (`burgers`, `accompagnements`, `condiments`).
- README mis à jour (arborescence réelle du projet).

---

5. Écarts avec le contrat `contrat-api/openapi.yaml` actuel (vérifié le 29/10/2026)

Point vérifié explicitement à la demande de l'équipe : `contrat-api/openapi.yaml` **n'est pas remplacé par Swagger UI**. Les deux coexistent avec des rôles différents :

- `contrat-api/openapi.yaml` est le contrat **écrit à l'avance** (design-first), versionné et revu par le lead Backend avant merge (voir README, section "Le contrat d'API"). C'est le seul endroit où sont documentées les routes pas encore codées.
- Swagger UI (`/api/v1/swagger-ui.html`) est **générée automatiquement depuis les annotations du code** (`springdoc-openapi-starter-webmvc-ui`, voir `OpenApiConfig.java`). Elle ne documente donc, à ce stade, que ce qui est réellement implémenté : `AuthController`, `StaffUserController`, `HealthController`. Il n'existe **aucun contrôleur** pour Produits, Commandes ou Paiements aujourd'hui — ces routes ne sont donc visibles que dans le fichier `openapi.yaml` statique, pas encore dans Swagger.

Autrement dit : supprimer `openapi.yaml` supprimerait la seule trace écrite du contrat `/produits`, `/commandes`, `/paiements` avant que ces contrôleurs n'existent. Il a donc été conservé.

En comparant ce fichier avec les sections 1 et 2 de ce document, les écarts suivants ont été identifiés (à faire trancher/mettre à jour par le lead Backend, conformément à la règle de revue du README) :

Élément	Dans `openapi.yaml` aujourd'hui	Dans ce document
`StatutCommande`	7 valeurs, `ANNULEE` absent	8 valeurs, `ANNULEE` inclus (voir table de transitions §2)
`Commande` / `CommandeCreation`	pas de `canal`	`canal: APP | COMPTOIR` (distingue commande App Client vs saisie comptoir)
`Commande` / `CommandeCreation`	pas de `moyenPaiement`	`moyenPaiement` avec les 5 clés Manager (`wave`, `orange-money`, `mixx-yas`, `carte`, `especes`)

Élément	Dans `openapi.yaml` aujourd'hui	Dans ce document
`LigneCommande`	`produitId` + `quantite` seulement	`+ nom, prixUnitaire` snapshotés (voir décision prix figé, §1.1)
`CommandeCreation`	`adresse` : string libre, pas de calcul de frais	`adresseLivraison` + `fraisLivraison` calculé depuis `Region`/`Secteur` (§1.3)
—	pas de champs	présents sur `Commande` (§1.1) `clientPrenom`/`clientNom`/`note`
—	aucun endpoint `/regions`, `/secteurs`	voir §2 "Zones de livraison"
—	aucun endpoint `/plats-du-jour`	voir §2
—	aucun endpoint `/accompagnements-condiments`	voir §2
—	aucun endpoint `/nouveautes`	voir §2 (déjà consommé côté Client)

Ce document est une annexe de travail pour alimenter la prochaine mise à jour d'`openapi.yaml` — il ne remplace pas le fichier et ne doit pas être traité comme le contrat final.